



COM3021/COMM115

Data Science at Scale

Reliable, scalable and maintainable data
systems and applications

DR HUGO BARBOSA

Agenda

- Previous lecture:
How and why data became so important for society?
- Today:
Challenges of data-intensive applications
Reliability
Scalability
Maintainability

Theses slides contain content from:

- Martin Kleppmann - Designing Data-Intensive Applications. The Big Ideas Behind Reliable, Scalable and Maintainable Systems (2017, O'Reilly)
- Manoochehri, Michael. Data just right : introduction to large-scale data & analytics

Data intensive applications

- In the past:
 - Single server solutions (mainframes)
 - Static (in terms of format)
 - Stored in expensive relational database management systems (RDBMs) like MS SQL and Oracle
-

Data intensive applications

- Things started to change with the popularisation of the Web
 - More companies and services being available online*
 - Dynamically-generated content*
 - Rapid growth (in terms of content and users)*
 - Google
 - ebay
 - Facebook
-

Data intensive applications

- Limitations of the traditional single-server approach

Availability and fault tolerance

Scalability (synchronisation, consistency)

Maintenance

The trade-offs

- Data consistency vs service availability
- Single powerful system vs several cheap commodity computers



Data-intensive applications

- Built from some fundamental functionalities
 - *Store data so it can be retrieved later (database)*
 - *Store the results of expensive operations to be used again soon (caching)*
 - *Allow users to efficiently search the data (indexing)*
 - *Periodically run specific routines on large amounts of accumulated data (batch processing)*
-

Data-intensive applications

- Each of these blocks can be implemented in many different ways.
- Each approach/solution to use has to be determined by the problem

E.g. Spatial data requires specialised data approaches

Representation: e.g., GeoJSON

Storage: PostGIS

Indexing: r-tree

At the end of the day...

- Reliable
 - Scalable
 - Maintainable
-

Reliability

Reliability

- The application **performs** the function that the user expected.
- It can tolerate the user making mistakes or using the software in unexpected ways.
- Its performance is good enough for the required use case, under the expected load and data volume.
- The system prevents any unauthorized access and abuse.

Resilient systems

Fault-tolerance and resilience

- Fault

When one component of the system works in an unexpected way

Hardware

Software

- Failure

When the entire system stops providing the service.

Hardware faults

- System failures frequently are related to faulty HDDs, memory modules, power supply units,
- In large data centres, these faults will happen all the time
e.g., a cluster with 10,000 HDDs will have on average one dead disk every day
- Traditionally we used hardware measures to prevent **system failures**
e.g., RAID for HDDs, redundant PSUs, hot-swappable CPUs
- However, with even larger infrastructures, even with all these measures in place, systems will fail

Necessity for infrastructures and solutions who are resilient even to individual system failures

Software faults

- Harder to anticipate

Hardware faults happen with a probability but most of the time are uncorrelated

- Can be present on many (if not all) nodes of a system causing widespread system failures

A faulty monitoring software

A process that is overusing resources

Cascading effect

Software faults

- The 2024 CrowdStrike software bug outage
- 8.5 million systems crashed
- Estimated £4.1bn in financial losses

2024 CrowdStrike-related IT outages



Multiple [blue screens of death](#) caused by a faulty software update on [baggage carousels](#) at [LaGuardia Airport, New York City](#)

Date	19 July 2024; 14 months ago
Location	Worldwide
Type	IT outage , computer crash
Cause	Faulty CrowdStrike software update
Outcome	~8.5 million Microsoft Windows operating systems crashed worldwide, causing global disruption of critical services

Scalability

Scalability

- A system's ability to cope with increased load

How do we define load?

Load

- *Load can be represented in terms of certain parameters specific to a system domain.*
 - Web application: requests per second
 - Database: read/write ratio
 - Online game: number of concurrent players
 - Bottlenecks
 - Average x extreme cases
 - Costs of different operations
-

Load

- Twitter as an example, using data published in November 2012. Two of Twitter's main operations are:

Post tweet

- A user can publish a new message to their followers (4.6k requests/sec on average, over 12k requests/sec at peak).

Home timeline

- A user can view tweets posted by the people they follow (300k requests/sec).
-

Load

- Twitter as an example, using data published in November 2012 [16]. Two of Twitter's main operations are:

Post tweet

- A user can publish a new message to their followers (4.6k requests/sec on average, over 12k requests/sec at peak).

Home timeline

- A user can view tweets posted by the people they follow (300k requests/sec).

- Fan-out

each user follows many people, and each user is followed by many people.

Performance

- Performance parameter
 - *Web services: response time (time between a user sending a request and receiving a response)*
 - *Data analysis: number of records per second or the time it takes to process a dataset of a certain size*
 - What happens to your system when load increases?
 - *Keep the system: how this increase will affect performance?*
 - *Keep the performance: how much you have to increase the resources?*
-

Performance

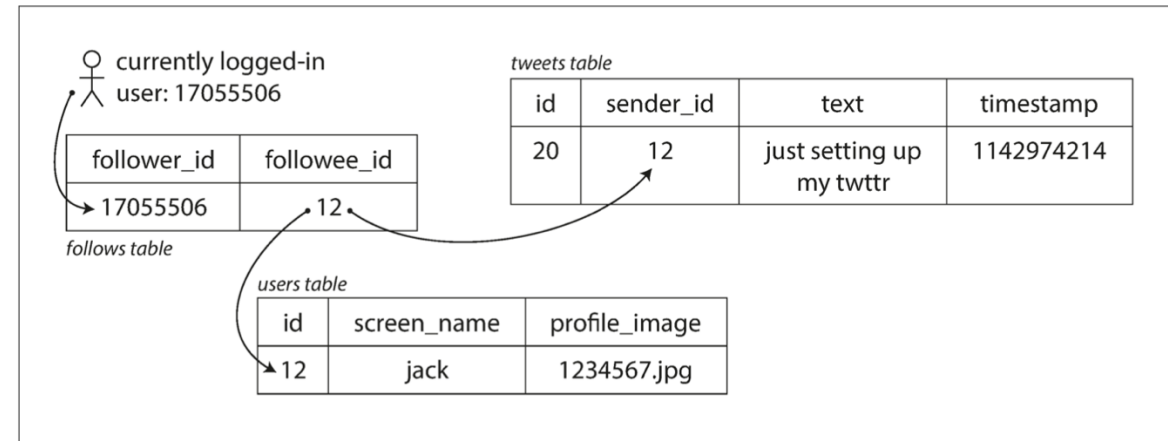
- Performance values are dynamic and can vary widely even for a same request
Current system load, network delays
 - Not only a single value (e.g., average response time) but the distribution of values is also important.
 - *What fraction of users are experiencing a performance level within the expected range?*
-

Twitter – approach 1

- Posting a tweet simply inserts the new tweet into a global collection of tweets.

When a user requests their home timeline, look up all the people they follow, find all the tweets for each of those users, and merge them (sorted by time).

- Performance: Twitter aims at delivering a tweet to the followers within five seconds

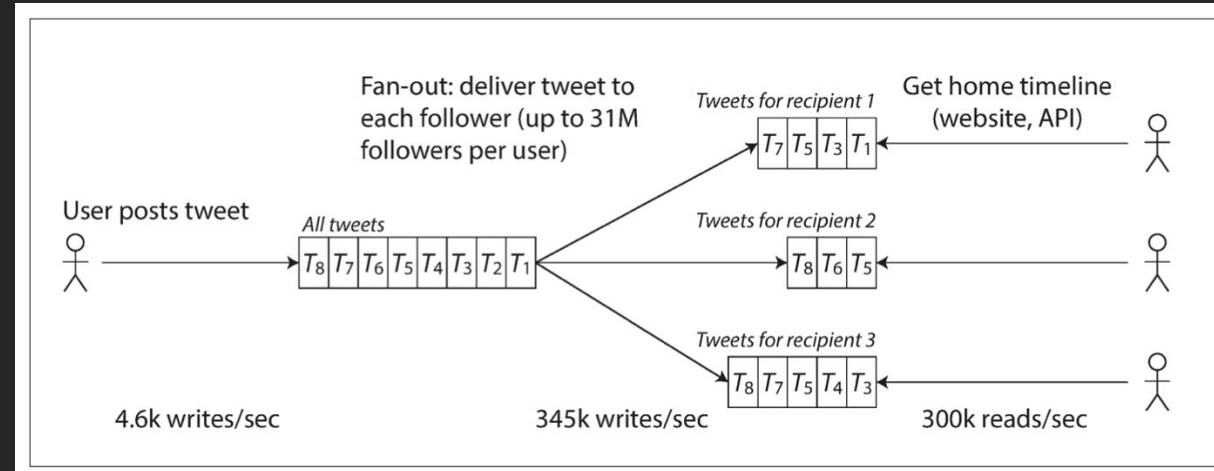


```
SELECT tweets.*, users.* FROM tweets
JOIN users ON tweets.sender_id = users.id
JOIN follows ON follows.followee_id = users.id
WHERE follows.follower_id = current_user
```

Twitter – Approach 2

- Maintain a cache for each user's home timeline—like a mailbox of tweets for each recipient user

When a user posts a tweet, look up all the people who follow that user, and insert the new tweet into each of their home timeline caches.



Which one would you
choose?

...and why?

Twitter

- First Twitter used approach 1, but the systems struggled to keep up with the load of home timeline queries
 - *2 orders of magnitude higher than tweets*
- Switched to approach 2
 - *On average, a tweet is delivered to about 75 followers,*
 - 4.6k tweets per second become 345k writes per second to the home timeline caches



Twitter

- Hybrid approach

For most users, Twitter uses approach 2 but for the users with very large number of followers, Twitter uses approach 1.

Scaling

- Vertical scaling (i.e., scaling up or shared-memory architectures)

The costs do not scale linearly

Limited fault tolerance

- Horizontal scaling (i.e., scaling out or shared-nothing architectures)

Costs can scale better

Better fault tolerance

Maintainability

Maintainability

- The maintainability of a system is the overall cost to maintain a system operational and updated

Operability

- How easy it is for the operation team to keep it running

Simplicity

- How easy it is for new people working on the system to understand it

Evolvability

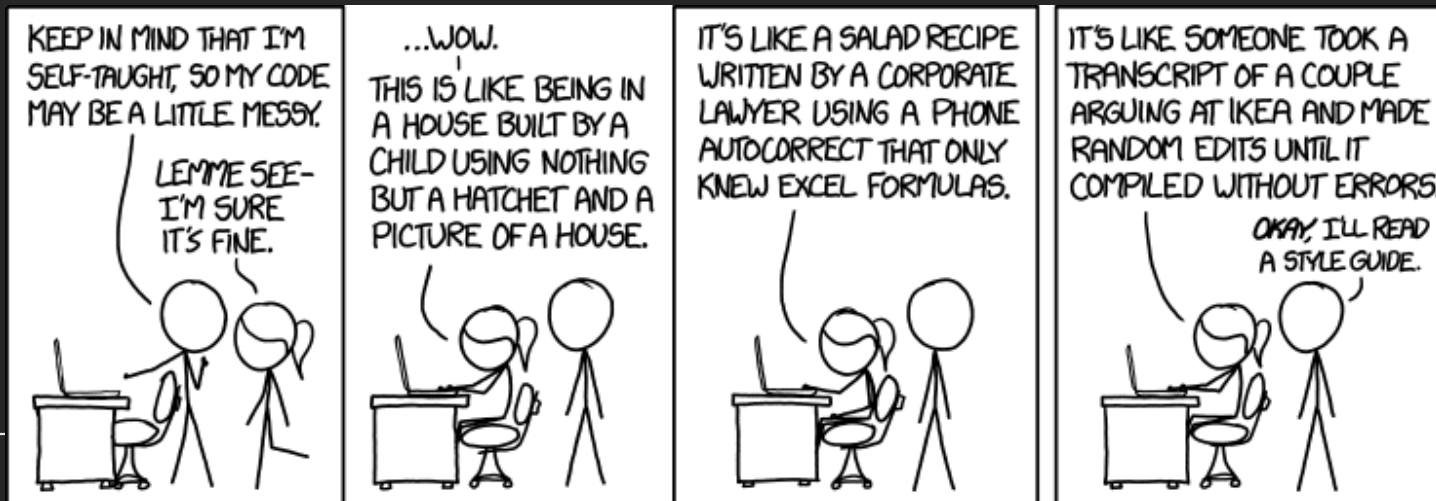
- How easy it is for engineers to make changes and update to the system (e.g., for new requirements, for increased load)
-

Operability

- Providing visibility into the runtime behaviour and internals of the system, with good monitoring
 - Providing good support for automation and integration with standard tools
 - Avoiding dependency on individual machines
 - Exhibiting predictable behaviour, minimising surprises
-

Simplicity

- Making a system simpler does not necessarily mean reducing its functionality
Often means removing accidental complexity
if it is not inherent in the problem that the system solves but arises only from the implementation.
- e.g., inconsistent architecture, obscure dependencies, poor styling or documentation
- Abstraction is frequently a good strategy



Evolvability

- The ease with which you can modify a data system
- closely linked to its simplicity and its abstractions

